

Relatório 13 - Prática: Predição e a Base de Aprendizado de Máquina (II)

Matheus Beuren Carvalho

1. Introdução

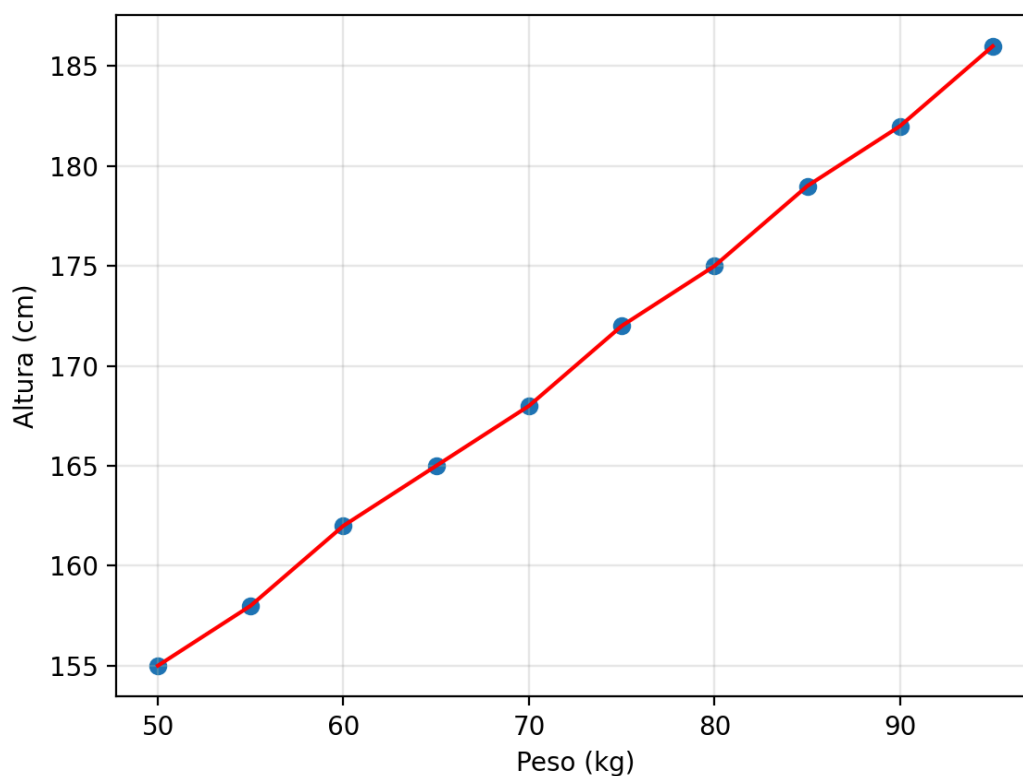
O desenvolvimento do card 13, gira em torno do aprendizado de máquina e predição. Ao longo do relatório não tive nenhuma dificuldade, muito interessante como podemos fazer uma máquina aprender padrões que as pessoas às vezes não conseguem enxergar nos dados. O aprendizado de máquina veio para ficar, e neste relatório descrevo os principais conceitos que aprendi.

2. Desenvolvimento

Regressão Linear

A regressão linear nada mais é do que uma reta traçada entre duas features, onde você pode prever um valor desconhecido por meio dessa linha. Um exemplo é a altura e peso como mostra a Figura 1.

Figura 1 - gráfico de regressão linear



O gráfico acima, segue um exemplo utilizado em aula, onde os pontos azuis são os dados observados e a regressão linear faz um linha com base nesses dados. Uma predição a ser feita é, por exemplo, quanto mede uma pessoa que tem 120kg.

O método padrão é o Ordinary Least Squares(mínimos quadrados ordinários), seu objetivo é minimizar a soma dos erros ao quadrado entre o ponto observado e a reta. A métrica utilizada é o R-squared(R^2), ela mede o quão boa é a sua reta, sendo medida de 0 a 1. O 0 é considerado péssimo, não explicando a variância dos dados, e 1 ótimo, pega toda a variância

Regressão Polinomial é a generalização da regressão linear. Ao invés de medir a reta para os dados observados, aqui é realizado por curvas e a cada grau de polinômio aumentado, mais complexa fica as curvas e a chance de overfitting, não pelos dados mas pela complexidade da fórmula.

As regressões múltiplas, são usadas quando você tem várias features para prever um único valor, onde diferente das regressões anteriores que só trabalhavam com uma.

Existem dois tipos de aprendizado em Machine Learning, o aprendizado supervisionado e o não supervisionado.

- Não supervisionado: O algoritmo recebe dados sem rótulos de resposta, ou seja, ele recebe somente os dados de treino e tenta achar padrões e os agrupa pelos padrões encontrados. Ex: Achar quais grupos de clientes de um site tende a comprar mais na loja;
- Supervisionado: O modelo recebe dados rotulados após o treino. Ex: fever o preço de um carro pelas suas características.

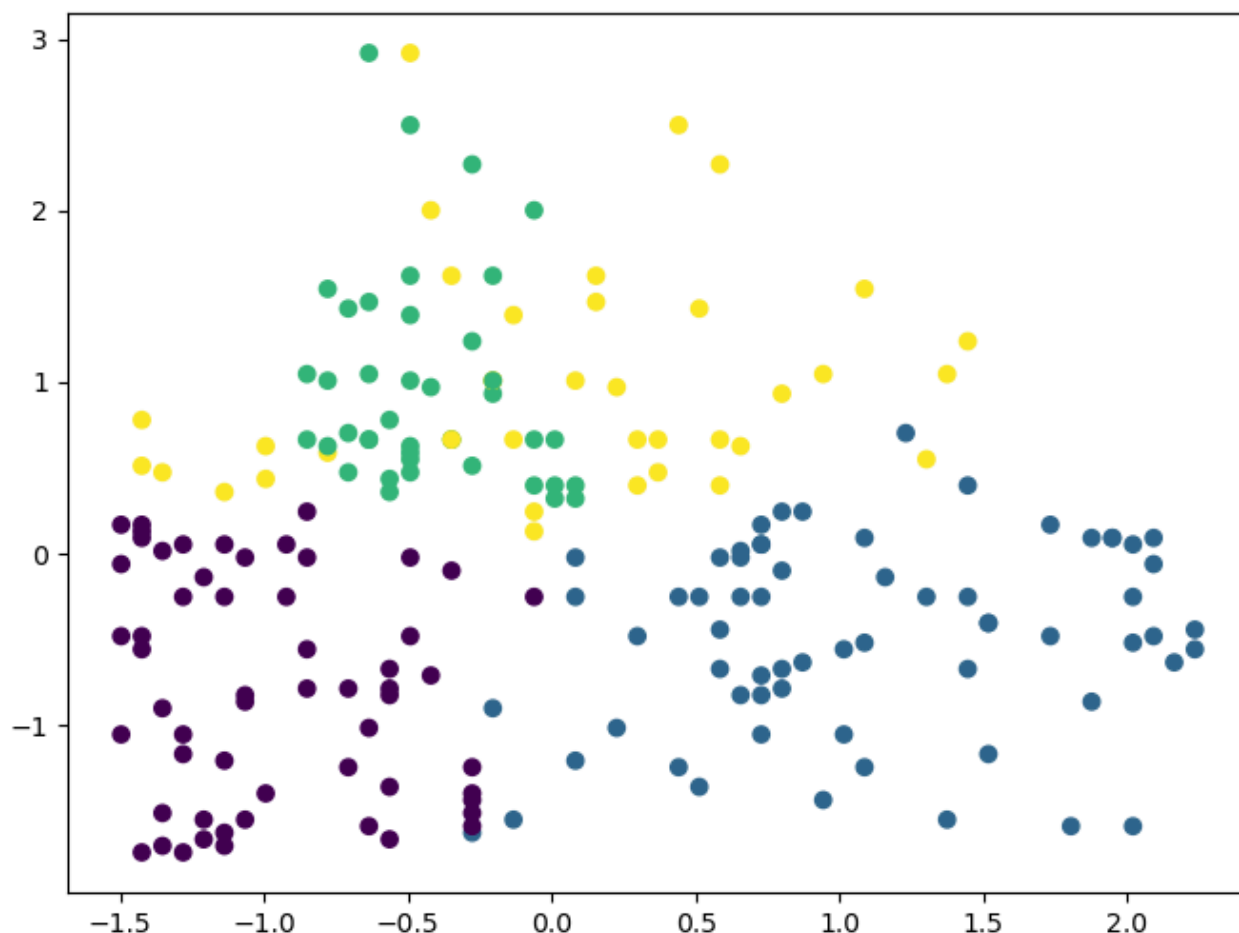
Nesse contexto, a divisão dos dados de treino e teste, afeta diretamente os resultados do modelo. O modo mais usado é o 'train_test_split' do sklearn, onde você separa o dataset em duas partes, uma para o treino do modelo e outro para testes, ou seja, evaluating que mede com métricas quão bem o modelo foi nos testes e quanto consegue prever. Mas essa separação, pode ter o azar ou coincidência de pegar um padrão que afete o modelo ou que poderia ser bom para o treino. Aí surge o K-cross-validation, que separa o dataset em K partes, treinando o modelo em cada uma delas, e ao final fazer uma média de desempenho(R^2) das partes treinadas.

O método Bayesian usado nas aulas do curso, é introduzido ao problema de spam e não spam de emails, para isso ele utiliza uma fórmula que calcula a probabilidade de um email ser spam dada uma condição, no email ter a palavra "free". Após a finalização do exercício de prática desta aula, tentei utilizar o método de divisão de dados 'train_test_split' do sklearn para ver se há alguma diferença nos resultados ao fazer esta divisão. Para avaliar os resultados, utilizei uma função do

sklearn que não foi abordada nos cursos do bootcamp ainda, mas eu já utilizei em projetos pessoais, que é o 'classification_report', nele você dá como parâmetro seu target(coluna target) comparando os resultados reais com os que o modelo previu nos dados de teste.

O algoritmo K-Means é uma forma de clustering, ou seja, um agrupamento de dados em categorias ou melhor clusters. Ele é um aprendizado não supervisionado, nisso ele não recebe dados rotulados(respostas do treino), então seu objetivo consiste em agrupar dados cujo seu centro é um centróide, nisso os dados próximos a este centróide são denominados daquela categoria. Como limitação, o algoritmo não descobre o número certo de clusters, mas sim o é definido como parâmetro, monitorando o erro quadrático, buscando o ponto em que aumentar K clusters deixa de reduzir esse erro de forma significativa. A seguir na **Figura 2**, mostrou o resultado obtido no notebook de prática com um dataset que utilizei do kaggle.

Figura 2 - Gráfico de dispersão com clustering = 4.



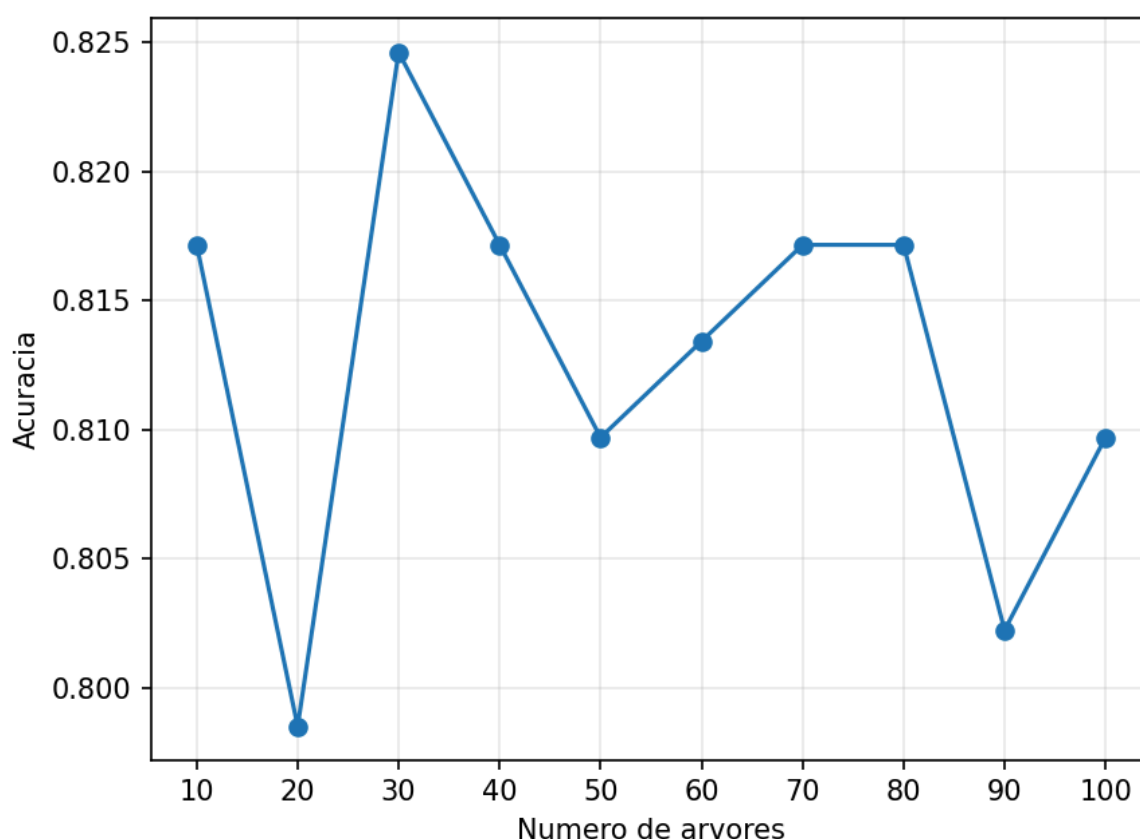
A entropia é uma métrica que mede o se o dataset é desordenado. Se ele tem classes e todos os dados estão em uma única classe então a entropia é

baixa(ordem), e quando a várias classes e dados distribuídos entre elas então a entropia é alta(desordem).

As árvores em Machine Learning é um modelo de aprendizado supervisionado. Elas são utilizadas quando a decisão a ser prevista depende de múltiplos atributos que influenciam o resultado. O seu principal problema é o overfitting(modelo não consegue prever os dados de teste como os de treino), por isso é utilizado o 'Random Forest', é um modelo que cada árvore de decisão é atribuído dados aleatórios(dados de treino), isso combate o overfitting pois o modelo é apresentado a mais dados diferentes, fazendo assim ele não decorar e também estar apto a prever dados não vistos ainda.

Para a prática desta aula de Árvores, utilizei um dataset do kaggle(Titanic) para implementar os conhecimentos da aula, e o objetivo era prever quais pessoas sobreviveram ou não. Nisso testei o modelo com os mesmos dados para diferentes quantidades de árvores, como pode ser observado na **Figura 3**, onde tem as acurácias por quantidade de árvores testadas.

Figura 3 - Gráfico de acurácias por árvores



Considerei os resultados como OK, poderiam ser melhores mas durante a escolha de features tive que excluir uma por conta de 70% ser NaN e não optei por

fazer dados artificiais pois ficaria superficial. Nisso o melhor parâmetro foi com 30 árvores, com 0.8246268656716418 de acurácia.

O XGboost é um modelo parecido com o Random Forest, também utiliza múltiplas árvores de decisão, mas a diferença está em como essas árvores se relacionam entre si. No Random Forest, as árvores são independentes, cada uma treinada com uma amostra aleatória diferente dos dados, no XGBoost, as árvores são encadeadas em sequência: cada nova árvore é construída para corrigir os erros da árvore anterior. Na parte prática utilizei o modelo XGBoost para prever se o câncer de mama é benigno ou maligno, através de uma dataset do kaggle. Com a utilização do modelo e parâmetros específicos, foram testadas diversas combinações de parâmetros para achar o modelo que tivesse a melhor acurácia e não tivesse overfitting, o resultado obtido foi ótimo, tendo uma acurácia de 95% dos dados de teste, sendo assim um modelo muito bom. Não foi aprofundada a procura de overfitting mas pelos testes feitos não teve nenhum indício de tal.

O SVM(Support Vector Machine) é uma técnica supervisionada utilizada para datasets com muitas dimensionalidades, ou seja, muitas features. O SVM permite configurar diferentes kernels (linear, polinomial, entre outros), cada um com custos computacionais e complexidade distintos. Utilizei o mesmo dataset da prática de XGBoost, já que ele possui muitas features(30 no total), o que se alinha com o uso recomendado do SVM. Testei três kernels diferentes, o linear, polinomial e rbf. O kernel linear obteve o melhor resultado, com 96% de acurácia.

3. Conclusão

O conteúdo deste card é essencial para entender e principalmente praticar os os conceitos de aprendizado de máquina. O que mais contribuiu para o meu aprendizado, foi aplicar os conceitos da aula a datasets do kaggle, sem resposta pronta. Um exemplo de prática foi ao testar o XGBoost, que resultou uma acurácia de 100% no treino, fui diminuindo os parâmetros (simplificando) para confirmar que o modelo não possui overfitting. Contudo, este card foi essencial para o meu aprendizado e os conceitos aprendidos são muito importantes.